

ASCIA

SMART HOME SYSTEMS

SPECIFICATIONS

ASCIA Mobile Application

Flutter iOS & Android — Complete Platform

Property	Value
Document Reference	CDC-2026-ASCIA-MOBILE-001
Version	1.0 — February 2026
Status	Confidential — Internal Use
Author	ASCIA Product Team
Technology	Flutter + Dart — iOS & Android
Target Platforms	iOS 15+, Android 10+ (API 29+)
Companion System	ASCIA OS v1.0+ (Home Assistant OS fork)

All rights reserved — ASCIA Inc. © 2026

Table of Contents

Table of Contents.....	2
Section 1 — Vision & Strategic Positioning	5
1.1 Purpose of This Document	5
1.2 Why Flutter (Not the Home Assistant Approach).....	5
1.3 Application Architecture Overview	6
Section 2 — Functional Modules.....	7
2.1 Module M1 — Authentication & Onboarding	7
2.1.1 First Launch Experience.....	7
2.1.2 Authentication Methods.....	7
2.1.3 Session Management	7
2.2 Module M2 — Core Dashboard (WebView)	8
2.2.1 WebView Integration	8
2.2.2 JavaScript-Dart Bridge	8
2.2.3 3D Plan Rendering on Mobile.....	9
2.3 Module M3 — LiDAR 3D Room Scanning	10
2.3.1 Overview.....	10
2.3.2 Compatible Devices	10
2.3.3 Scanning Process	10
2.3.4 Technical Requirements.....	10
2.3.5 Fallback for Non-LiDAR Devices	11
2.4 Module M4 — NFC Device Pairing	12
2.4.1 Overview.....	12
2.4.2 Pairing Process.....	12
2.4.3 Technical Requirements.....	12
2.5 Module M5 — Push Notifications.....	12
2.5.1 Notification Architecture	12
2.5.2 Technical Requirements.....	13
2.6 Module M6 — Geofencing & Location Services	13
2.6.1 Overview.....	13
2.6.2 Geofence Zones.....	13
2.6.3 Technical Requirements.....	14
2.7 Module M7 — BLE & UWB Presence	15
2.7.1 Bluetooth Low Energy (BLE) Presence	15
2.7.2 Ultra-Wideband (UWB) Precision Location.....	15
2.8 Module M8 — Camera & Intercom.....	15

2.8.1 Live Camera Viewing	15
2.8.2 Two-Way Intercom	16
2.9 Module M9 — Offline Mode & Local Cache	16
2.9.1 Cached Dashboard	16
2.9.2 Local Network Priority	16
2.10 Module M10 — Apple Watch & Wear OS	17
2.10.1 Watch App Features	17
2.10.2 Technical Requirements.....	17
Section 3 — Non-Functional Requirements & KPIs	18
3.1 Performance	18
3.2 Security	18
3.3 Compatibility & Platform Requirements.....	19
3.4 Accessibility	19
3.5 Localization	19
Section 4 — Technical Architecture	20
4.1 Project Structure.....	20
4.2 State Management	20
4.3 Navigation	21
4.4 Communication with ASCIA OS.....	21
4.5 Native Plugin Architecture	21
Section 5 — User Interface Design	23
5.1 Design Principles.....	23
5.2 Native Screens (Not in WebView).....	23
5.3 Navigation Structure	23
Section 6 — Development Roadmap	25
6.1 Phase Alignment with ASCIA OS.....	25
6.2 MVP Scope (What Is Included).....	25
6.3 Post-MVP Scope (What Is Excluded from MVP).....	25
Section 7 — Testing Strategy.....	27
7.1 Test Coverage Requirements.....	27
7.2 Device Testing Matrix	27
7.3 Hardware Integration Testing.....	27
Section 8 — App Store Deployment & Distribution.....	29
8.1 App Store Requirements	29
8.2 CI/CD Pipeline	29
8.3 Update Strategy.....	29

Section 9 — Glossary31

Section 1 — Vision & Strategic Positioning

1.1 Purpose of This Document

This document defines the complete functional and technical specifications for the ASCIA Mobile Application, the primary customer-facing touchpoint for the ASCIA smart home ecosystem. This application is a companion to ASCIA OS (the server-side home automation operating system) and is designed to provide full remote control, monitoring, configuration, and advanced native capabilities that extend beyond what a web browser can deliver.

The ASCIA Mobile Application is NOT a simple WebView wrapper around the ASCIA web interface. It is a hybrid native application that combines the full ASCIA web dashboard (rendered via an embedded WebView) with deep native platform integrations that require direct access to device hardware and operating system APIs.

1.2 Why Flutter (Not the Home Assistant Approach)

The Home Assistant Companion App uses a native iOS/Android approach (Swift/Kotlin) with a WebView that loads the Lovelace web interface. This approach is adequate for a technical audience that accepts a web-wrapped experience. ASCIA targets a premium non-technical market and requires a fundamentally different mobile strategy.

Criteria	HA Companion Approach	ASCIA Flutter Approach
Core UI	WebView loading Lovelace (HA web UI)	WebView loading ASCIA SvelteKit UI (custom branded)
Native Features	Sensors (GPS, battery, steps), notifications	LiDAR 3D scan, NFC pairing, UWB/BLE presence, AR, biometrics, deep notifications
Platform Plugins	None — no custom native plugins	Custom Swift/Kotlin plugins for hardware access
Watch Support	Basic iOS widget only	Full Apple Watch + Wear OS companion apps
Offline Mode	No offline capability	Cached dashboard, local scene triggers, queued commands
Target User	Technical enthusiast	Non-technical premium customer
Branding	Home Assistant branding visible	100% ASCIA branding — zero HA visibility
Codebase	Separate iOS + Android codebases	Single Flutter codebase — reduced maintenance cost

Flutter was selected for the following strategic reasons:

- Single codebase for iOS and Android, reducing development and maintenance cost by approximately 40%.

- Native plugin architecture (Platform Channels) allows writing Swift and Kotlin code that integrates seamlessly with the Dart layer.
- flutter_inappwebview provides a production-grade WebView with full bidirectional JavaScript-Dart communication (JS injection, message handlers, cookie management).
- Flutter supports Apple Watch (watchOS) and Wear OS companion apps from the same project.
- The Flutter ecosystem has mature packages for ARKit/ARCore, NFC, BLE, UWB, biometrics, and background processing.

1.3 Application Architecture Overview

The ASCIA Mobile App operates as a hybrid application with two distinct layers:

Layer	Technology	Responsibility	Percentage of App
WebView Layer	ASCIA SvelteKit frontend via flutter_inappwebview	Dashboard, 3D plan, cameras, automations, device control, settings UI	~70-75% of user interactions
Native Layer	Flutter + Dart + Swift/Kotlin plugins	LiDAR scan, NFC, BLE/UWB presence, push notifications, biometrics, geofencing, offline cache, watch apps	~25-30% of user interactions

The two layers communicate via a bidirectional JavaScript-Dart bridge. The SvelteKit frontend can invoke native functions (e.g., trigger LiDAR scan, request BLE status) and the native layer can push data into the WebView (e.g., GPS coordinates, NFC scan results, notification taps).

Section 2 — Functional Modules

2.1 Module M1 — Authentication & Onboarding

2.1.1 First Launch Experience

The first launch must be seamless for a non-technical customer who has just received their ASCIA server.

1. User opens the ASCIA app for the first time.
2. The app scans the local network for ASCIA OS servers via mDNS/Bonjour discovery.
3. If a server is found on the local network, the app connects automatically and displays the setup wizard.
4. If no server is found (user is remote), the app prompts for the ASCIA Cloud connection or manual WireGuard VPN configuration.
5. The user creates their account (or logs in if the server was pre-configured by an ASCIA technician).
6. Biometric authentication setup (Face ID, Touch ID, fingerprint) is offered immediately.
7. The app downloads and caches the dashboard for offline access.

2.1.2 Authentication Methods

Method	Implementation	Use Case
Biometric (primary)	Face ID (iOS), Fingerprint (Android) via local_auth plugin	Default quick access after initial setup
PIN Code	6-digit PIN stored in secure enclave (Keychain/Keystore)	Fallback when biometric fails
Username + Password	Standard HA authentication via token	Initial setup and remote connections
ASCIA Cloud SSO	OAuth 2.0 via ASCIA Cloud relay servers	Remote access without VPN
WireGuard VPN	Auto-configured VPN profile installed on device	Fully local access from anywhere

2.1.3 Session Management

- Authentication token stored in iOS Keychain / Android Keystore (hardware-backed secure storage).
- Token auto-refresh with silent re-authentication when possible.
- Automatic session expiry after 30 days of inactivity (configurable by admin).
- Multi-device support: the user can be logged in on phone, tablet, and watch simultaneously.
- Remote session revocation: admin can force-logout any device from the ASCIA OS interface.

2.2 Module M2 — Core Dashboard (WebView)

2.2.1 WebView Integration

The core dashboard is the ASCIA SvelteKit frontend loaded inside flutter_inappwebview. This is NOT a simple URL load — it requires deep integration:

- Authentication token injection: the native layer injects the HA Bearer token into the WebView via JavaScript before the page loads.
- Cookie synchronization: session cookies are managed by the native layer to prevent re-login on each app launch.
- Custom User-Agent: the WebView identifies itself as the ASCIA Mobile App so the SvelteKit frontend can adapt its layout and behavior (e.g., hide elements that are native, add mobile-specific touch targets).
- Pull-to-refresh: native gesture triggers a full page reload.
- Deep link handling: push notification taps open the correct page/entity in the WebView.
- Error handling: if the WebView fails to load (server offline, network error), the app displays a native error screen with cached dashboard and retry option.

2.2.2 JavaScript-Dart Bridge

The bidirectional bridge is the critical integration layer. It allows the web frontend to invoke native capabilities and vice versa.

Direction	Channel Name	Payload	Use Case
Web → Native	ascia.startLidarScan	{quality: high/medium}	User taps 'Scan Room' in the 3D plan editor
Web → Native	ascia.startNfcScan	{timeout: 30}	User taps 'Add Device via NFC'
Web → Native	ascia.requestLocation	{}	Automation needs current GPS position
Web → Native	ascia.triggerHaptic	{type: success/warning/error}	Feedback on device control actions
Web → Native	ascia.openNativeSettings	{section: notifications/vpn/biometric}	Deep link to native settings screens
Native → Web	ascia.onLidarComplete	{modelUrl: string, roomCount: int}	LiDAR scan finished, 3D model ready
Native → Web	ascia.onNfcDetected	{deviceId: string, type: string}	NFC tag scanned, device identified
Native → Web	ascia.onGeofenceEvent	{event: enter/exit, zone: string}	User entered or left a geofence zone
Native → Web	ascia.onBlePresence	{userId: string, room: string, rssi: int}	BLE beacon detected user in room

Native → Web	ascia.onNotificationTap	{entityId: string, action: string}	User tapped a notification action
-------------------------	--------------------------------	---	--

2.2.3 3D Plan Rendering on Mobile

The ASCIA 3D plan (Three.js/WebGL) runs inside the WebView on mobile. Performance targets:

KPI	Target	Condition
3D framerate	30-60 FPS	iPhone 12 Pro / Pixel 6 or equivalent
3D plan load time	< 3 seconds	Cached model, warm WebView
State update latency	< 150ms	Local WebSocket, simple entity change
GPU memory	< 200MB	Optimized compressed model
Touch interaction	< 50ms response	Tap to control, pinch to zoom, swipe to rotate

If the device GPU cannot sustain 30 FPS, the app automatically falls back to the 2D floor plan view with a notification explaining the limitation.

2.3 Module M3 — LiDAR 3D Room Scanning

2.3.1 Overview

The LiDAR scanning feature allows customers with compatible devices to scan their home and automatically generate a 3D model for the ASCIA 3D dashboard. This is one of the most technically demanding features and a key differentiator that justifies the native Flutter approach.

2.3.2 Compatible Devices

Platform	Requirement	Technology	Accuracy
iOS	iPhone 12 Pro or later / iPad Pro 2020+	ARKit + LiDAR sensor	< 1cm at 5m range
Android	Devices with depth sensor (Samsung S20 Ultra, Pixel 4+)	ARCore Depth API	< 3cm at 5m range
Fallback	Any device without LiDAR	ARKit/ARCore structure from motion (lower quality)	< 10cm at 3m range

2.3.3 Scanning Process

8. User opens the 3D Plan section and taps 'Scan Room with LiDAR'.
9. The app requests camera and motion sensor permissions (one-time).
10. The AR session starts with a visual guide overlay showing the user how to move their phone slowly around the room.
11. Real-time point cloud visualization shows the scanned area with color-coded quality indicators (green = good coverage, red = needs more data).
12. The user taps 'Complete Room' when the current room is fully scanned.
13. The app prompts: 'Scan another room?' — the user can scan the entire home room by room.
14. When all rooms are scanned, the app generates a 3D mesh model from the combined point clouds.
15. The model is converted to GLB/GLTF format and uploaded to the ASCIA OS server via the REST API.
16. ASCIA OS processes the model and makes it available in the 3D dashboard within 60 seconds.

2.3.4 Technical Requirements

- Native Swift plugin using ARKit SceneReconstruction API (iOS) for mesh generation.
- Native Kotlin plugin using ARCore Raw Depth API (Android) for point cloud capture.
- Real-time mesh simplification to keep the model under 50MB per room.
- Texture capture: the app captures room colors and materials for realistic rendering.
- Room boundary detection: automatic detection of walls, floors, and ceilings.
- Door and window detection: identified openings are marked for ASCIA zone assignment.

- Export format: GLB/GLTF with embedded textures, optimized for Three.js rendering.
- Upload with progress indicator and resume capability for large models.

2.3.5 Fallback for Non-LiDAR Devices

Devices without LiDAR hardware can still create a 3D model using two alternative methods:

- Manual 2D editor: the user draws rooms using the touch-based floor plan editor within the app (same as the ASCIA OS web editor, but optimized for touch input).
- Import: the user uploads a SweetHome 3D (.sh3d) or SketchUp (.skp) file from their device or cloud storage.

2.4 Module M4 — NFC Device Pairing

2.4.1 Overview

ASCIA hardware devices ship with an NFC tag and QR code. The mobile app enables instant device pairing by scanning either one, eliminating the need to navigate menus or enter identifiers manually.

2.4.2 Pairing Process

17. The user holds their phone near an ASCIA device (NFC) or points the camera at the QR code.
18. The app reads the device identifier and type from the tag/code.
19. The app sends a pairing request to ASCIA OS via the REST API.
20. ASCIA OS confirms the device is detected on the network and returns its current status.
21. The app displays a confirmation screen: device name, type, suggested zone assignment.
22. The user confirms or modifies the zone and name.
23. The device is fully integrated into ASCIA OS and appears in the 3D plan.

2.4.3 Technical Requirements

- iOS: Core NFC framework for reading NDEF tags in foreground. Background NFC reading via NFC tag reader session.
- Android: NFC adapter with NDEF discovery intent filter. Both foreground dispatch and background tag discovery supported.
- QR code scanning: native camera integration via `mobile_scanner` or similar Flutter plugin.
- ASCIA NFC tag format: NDEF record containing JSON payload with device UUID, device type, firmware version, and manufacturing date.
- Security: NFC payloads are signed with ASCIA device certificates to prevent spoofing.

2.5 Module M5 — Push Notifications

2.5.1 Notification Architecture

ASCIA notifications are categorized by criticality and deliver rich, actionable content. They are not simple text alerts — they include images, action buttons, and deep links to specific screens in the app.

Category	iOS Behavior	Android Behavior	Example
Critical Alert	Bypasses Do Not Disturb, plays loud sound, persistent banner	High-priority channel, heads-up notification, full-screen intent	Fire detected, water leak, intrusion alarm

Action Notification	Expandable with 2-3 action buttons	Action buttons in notification shade	Oven on 2 hours — [Turn Off] [Leave On]
Image Notification	Notification with attached image thumbnail	BigPictureStyle with camera snapshot	Motion detected at entrance + camera preview
Doorbell	Image + action buttons + intercom	Full-screen intent + image + talk button	Someone at door + thumbnail + [Talk] [Open] [Ignore]
System Report	Silent notification, badge update only	Low-priority channel, no sound	Weekly system health report available
Battery Alert	Standard notification	Standard notification	Office door sensor — battery at 10%

2.5.2 Technical Requirements

- iOS: Apple Push Notification service (APNs) with critical alert entitlement (requires Apple approval for critical alerts that bypass DND).
- Android: Firebase Cloud Messaging (FCM) with high-priority data messages for critical alerts.
- Notification channels (Android): separate channels for Critical, Security, Device, System, allowing per-category user control.
- Rich notifications: image attachments via Notification Service Extension (iOS) and BigPictureStyle (Android).
- Action buttons: up to 3 actions per notification, each triggering an ASCIA OS REST API call.
- Deep linking: each notification contains an entity ID and screen identifier. Tapping opens the app directly to the relevant device, camera, or automation.
- Notification grouping: multiple alerts from the same zone or event are grouped into a summary notification.
- Quiet hours: user-configurable schedule to suppress non-critical notifications (configured in-app, enforced server-side).
- Delivery confirmation: the app sends a delivery receipt to ASCIA OS so unacknowledged critical alerts can be escalated (repeated every 5 minutes).

2.6 Module M6 — Geofencing & Location Services

2.6.1 Overview

Geofencing enables ASCIA OS to trigger automations based on the user's physical location relative to their home and registered addresses. This is critical for arrival/departure automations, commute notifications, and the Away security mode.

2.6.2 Geofence Zones

Zone Type	Radius	Trigger	Automation Example
-----------	--------	---------	--------------------

Home	100-200m (configurable)	Enter / Exit	Arrive home → disarm alarm, turn on lights. Leave home → arm alarm, energy mode.
Near Home	500m-2km	Enter	12 minutes away → preheat house, turn on entry lights.
Work	200m	Exit	Leave work → estimate arrival time, prepare house.
Custom	User-defined	Enter / Exit	Arrive at gym → start robot vacuum. Arrive at school → notify parent.

2.6.3 Technical Requirements

- iOS: Core Location with CLCircularRegion for geofencing (maximum 20 concurrent regions). Significant location change monitoring for battery efficiency. Always authorization required with clear purpose string.
- Android: Geofencing API via Google Play Services. Background location permission (ACCESS_BACKGROUND_LOCATION) with user explanation per Google Play policy.
- Battery optimization: use significant location changes (iOS) and activity recognition (Android) to minimize GPS polling. Target < 2% additional battery drain per day.
- Location data privacy: GPS coordinates are sent only to the user's own ASCIA OS server. No location data is transmitted to ASCIA Cloud or any third party.
- Waze/Google Maps integration: real-time traffic estimation for commute-based automations (API call from ASCIA OS server, not from the app directly).
- Multi-user support: each family member's phone reports its own location independently. ASCIA OS aggregates presence data to determine house occupancy.

2.7 Module M7 — BLE & UWB Presence

2.7.1 Bluetooth Low Energy (BLE) Presence

The user's phone acts as a BLE beacon, allowing ASCIA OS to detect room-level presence without requiring dedicated hardware badges.

- The app broadcasts a unique BLE advertisement containing the user's ASCIA ID.
- ASCIA BLE receivers (built into ASCIA sensors or dedicated beacons) detect the signal and report RSSI values to ASCIA OS.
- ASCIA OS triangulates the user's approximate room position based on multiple receiver readings.
- Accuracy: room-level (which room the user is in), not centimeter-level.
- Background operation: BLE advertising must continue when the app is backgrounded or the screen is locked.
- iOS limitation: Core Bluetooth background advertising is restricted. The app uses iBeacon region monitoring as a fallback.
- Android: BLE advertising in background is supported but subject to Doze mode restrictions. The app uses a foreground service with a persistent notification.

2.7.2 Ultra-Wideband (UWB) Precision Location

For premium installations, UWB provides centimeter-accurate indoor positioning.

- iOS: Nearby Interaction framework with UWB anchors (iPhone 11+).
- Android: UWB API (limited device support — Samsung Galaxy S21+, Pixel 6 Pro+).
- The app communicates with ASCIA UWB anchors installed in the home to determine precise position.
- Position data is visualized in the 3D plan as the user's avatar moving in real time.
- Person-specific automations: 'When Jean-Pierre enters the kitchen → start coffee maker with his profile.'
- UWB is a Phase 2 feature and is optional — the app must gracefully degrade to BLE when UWB hardware is not available.

2.8 Module M8 — Camera & Intercom

2.8.1 Live Camera Viewing

- View any ASCIA camera stream directly in the app via HLS (HTTP Live Streaming) from the ASCIA NVR.
- Multi-camera grid view: display 2, 4, or 9 cameras simultaneously.
- Full-screen mode with pinch-to-zoom on any camera.
- Timeline scrubbing: scroll through recorded footage with a visual timeline.
- Snapshot button: capture a still frame and save to the device photo library.
- Clip export: select a time range and download an MP4 clip to the device.

2.8.2 Two-Way Intercom

When a doorbell event is triggered or the user opens a camera with a speaker:

- The app establishes a WebRTC audio channel with the camera via the ASCIA NVR.
- Push-to-talk button: the user holds to speak, releases to listen.
- Full duplex option for cameras that support simultaneous audio.
- Echo cancellation and noise suppression applied natively before transmission.
- Latency target: < 500ms round-trip for natural conversation.

2.9 Module M9 — Offline Mode & Local Cache

2.9.1 Cached Dashboard

When the ASCIA OS server is unreachable (internet outage, user out of VPN range), the app does not display a blank screen. It provides a degraded but functional experience.

- The last-known state of all devices is cached locally in an SQLite database on the device.
- The cached dashboard displays device states with a visual indicator showing they are stale (timestamp of last update).
- Scenes can be queued: the user can trigger a scene, and the command is stored and executed when connectivity is restored.
- Camera feeds are unavailable offline (real-time streams cannot be cached).
- Automations continue to run on the ASCIA OS server independently of the app.

2.9.2 Local Network Priority

The app always prefers the local network connection over cloud/VPN:

- On launch, the app checks if the ASCIA OS server is reachable on the local network (mDNS discovery).
- If found locally, the app connects directly with minimal latency (< 10ms).
- If not found locally, the app falls back to ASCIA Cloud relay or WireGuard VPN.
- Automatic switchover: if the user arrives home while using the app remotely, the connection transparently switches to local.

2.10 Module M10 — Apple Watch & Wear OS

2.10.1 Watch App Features

Feature	Description	Priority
Quick Scenes	Activate predefined scenes (Home, Away, Night, Cinema) with a single tap	P0
Device Control	Toggle lights, locks, garage door from the watch	P0
Notifications	Mirror critical alerts with haptic feedback	P0
Camera Snapshot	View the last snapshot from the doorbell camera	P1
Presence Badge	The watch acts as a BLE/UWB presence badge for room-level detection	P1
Complications	Home status (armed/disarmed), temperature, door status on watch face	P1

2.10.2 Technical Requirements

- Apple Watch: watchOS 9+ using WatchConnectivity framework. Independent watch app with local cache for scenes and device states.
- Wear OS: Wear OS 3+ using Data Layer API. Tile support for quick scene access.
- Communication: the watch app communicates with the phone app, which relays commands to ASCIA OS. Direct server communication from the watch is not required for MVP.
- The watch app must function even if the phone app is killed, using cached data and queued commands.

Section 3 — Non-Functional Requirements & KPIs

3.1 Performance

KPI	Target	Measurement Condition
App cold start to dashboard	< 3 seconds	iPhone 12 / Pixel 6, local network, cached WebView
App warm resume	< 500ms	App returning from background
Local command latency	< 100ms	Device on local network, simple toggle command
Remote command latency	< 500ms	Via ASCIA Cloud or WireGuard VPN, 10 Mbps+ connection
3D plan framerate (mobile)	30-60 FPS	5-8 room plan, 50 entities, iPhone 12 Pro / Pixel 6
Camera stream startup	< 2 seconds	HLS stream from local ASCIA NVR
Push notification delivery	< 3 seconds	From ASCIA OS event to notification on device
LiDAR scan processing	< 30 seconds per room	iPhone 14 Pro, average room 15-25m ²
NFC pairing total time	< 10 seconds	From NFC tap to device visible in dashboard
Geofence trigger accuracy	< 100m	Standard GPS, outdoor conditions
Battery impact (background)	< 2% per day	Geofencing + BLE beacon active

3.2 Security

Requirement	Implementation
Authentication tokens	Stored in iOS Keychain / Android Keystore (hardware-backed)
Local data encryption	AES-256 encryption for cached dashboard data and credentials
Network communication	TLS 1.3 mandatory for all server communication
Certificate pinning	SSL certificate pinning for ASCIA Cloud connections
Biometric protection	Face ID / Touch ID / Fingerprint required to open app (configurable)
Session management	Automatic token expiry, remote session revocation by admin
Jailbreak/root detection	Warning displayed on compromised devices, optional block by admin policy
No third-party analytics	Zero tracking SDKs. No data sent to Google Analytics, Firebase Analytics, or any third party.

Screenshot protection	Optional: admin can disable screenshots in the app for commercial installations
------------------------------	--

3.3 Compatibility & Platform Requirements

Platform	Minimum Version	Recommended	Notes
iOS	iOS 15.0	iOS 17+	LiDAR requires iPhone 12 Pro+. UWB requires iPhone 11+.
Android	Android 10 (API 29)	Android 13+	UWB limited to Samsung S21+, Pixel 6 Pro+. NFC requires NFC hardware.
Apple Watch	watchOS 9.0	watchOS 10+	Series 5 or later recommended for performance.
Wear OS	Wear OS 3.0	Wear OS 4+	Requires Google Play Services.
Flutter SDK	3.16+	Latest stable	Null safety required. Dart 3.0+.

3.4 Accessibility

- WCAG AA contrast ratios for all text and interactive elements.
- VoiceOver (iOS) and TalkBack (Android) compatibility for all native screens.
- Dynamic text sizing: the native UI respects system font size preferences.
- Haptic feedback for all confirmations, errors, and critical alerts.
- Color-blind safe: critical status indicators use shape + color (not color alone).

3.5 Localization

- Primary languages: French (Quebec priority) and English.
- All native UI strings externalized in ARB files (Flutter internationalization standard).
- Date, time, and number formatting adapted to device locale.
- Right-to-left (RTL) layout support prepared for future Arabic market expansion.

Section 4 — Technical Architecture

4.1 Project Structure

The Flutter project follows a clean architecture pattern with clear separation between the presentation, domain, and data layers.

Directory	Content	Responsibility
lib/core/	App configuration, constants, theme, routing	Shared infrastructure
lib/features/dashboard/	WebView integration, bridge handlers	Core dashboard module
lib/features/auth/	Login, biometrics, session management	Authentication module
lib/features/lidar/	LiDAR scanning UI and state management	3D room scanning module
lib/features/nfc/	NFC/QR pairing flows	Device pairing module
lib/features/notifications/	Notification handling, channels, deep links	Push notification module
lib/features/location/	Geofencing, GPS, presence	Location services module
lib/features/ble/	BLE beacon broadcasting and scanning	Bluetooth presence module
lib/features/camera/	Camera viewing, intercom, clips	Camera and NVR module
lib/features/offline/	Local cache, command queue	Offline mode module
lib/features/watch/	Watch communication layer	Apple Watch / Wear OS module
ios/Runner/Plugins/	Swift native plugins (LiDAR, NFC, UWB)	iOS platform code
android/app/src/main/kotlin/plugins/	Kotlin native plugins (LiDAR, NFC, UWB)	Android platform code

4.2 State Management

The application uses Riverpod as its state management solution for the following reasons:

- Compile-time safety: providers are type-safe and catch errors at compile time.
- Dependency injection built-in: providers can depend on other providers without manual wiring.
- Async-first: seamless handling of API calls, WebSocket streams, and background events.

- Testability: every provider can be overridden in tests without changing production code.

4.3 Navigation

GoRouter handles all navigation with deep link support:

- Every screen has a unique URI path (e.g., /dashboard, /cameras/entrance, /device/sensor.door_bedroom1).
- Push notifications include a URI that GoRouter resolves directly to the correct screen.
- WebView navigation events (user taps a device in the 3D plan) trigger GoRouter navigation for native overlays.
- Authentication guard: unauthenticated users are redirected to login regardless of the deep link target.

4.4 Communication with ASCIA OS

Protocol	Use Case	Implementation
WebSocket (HA WS API)	Real-time device state updates, command execution	Persistent WebSocket connection managed by the native layer, shared with the WebView
REST API (HA REST + ASCIA)	One-off operations: LiDAR upload, device pairing, config changes	Dio HTTP client with token injection and retry logic
HLS (HTTP Live Streaming)	Camera live streams and recorded footage	Native video player (video_player or better_player plugin)
WebRTC	Two-way audio intercom	flutter_webrtc plugin for peer-to-peer audio
mDNS	Local server discovery	multicast_dns or nsd_android for network scanning
FCM / APNs	Push notifications from ASCIA OS	firebase_messaging plugin + custom APNs handler for critical alerts

4.5 Native Plugin Architecture

Each native capability that requires platform-specific code is implemented as a Flutter Platform Channel plugin:

Plugin Name	Platform Code	Dart Interface	Purpose
ascia_lidar	Swift (ARKit) + Kotlin (ARCore)	LidarService.startScan(), onProgress, onComplete	3D room scanning

ascia_nfc	Swift (CoreNFC) + Kotlin (NfcAdapter)	NfcService.startScan(), onTagDetected	NFC device pairing
ascia_uwb	Swift (NearbyInteraction) + Kotlin (UWB API)	UwbService.startRanging(), onPositionUpdate	UWB precision location
ascia_ble_beacon	Swift (CoreBluetooth) + Kotlin (BLE API)	BleBeacon.startAdvertising(), startScanning()	BLE presence broadcasting
ascia_critical_alert	Swift (UNNotification) + Kotlin (NotificationManager)	CriticalAlert.send(), configure()	Critical alerts that bypass DND

Section 5 — User Interface Design

5.1 Design Principles

- 100% ASCIA branding: no Home Assistant logo, terminology, or visual identity anywhere in the app.
- Mobile-first: all interactions designed for one-handed thumb use on smartphones.
- Dark mode default: optimized for AMOLED screens, with light mode available.
- Consistent with the ASCIA SvelteKit web interface design language (colors, typography, iconography, spacing).
- Animation budget: transitions < 300ms, no animation blocks user interaction.
- Haptic feedback: every toggle, confirmation, and error provides tactile response.

5.2 Native Screens (Not in WebView)

The following screens are implemented as native Flutter widgets (not WebView). They require native platform access or must work offline:

Screen	Reason for Native	Key Elements
Splash / Loading	First impression, must load before WebView	ASCIA logo animation, server connection status
Login / Biometric	Secure enclave access, biometric APIs	Face ID / fingerprint prompt, PIN fallback, server selector
LiDAR Scanner	ARKit/ARCore camera overlay	AR session, point cloud visualization, room completion UI
NFC / QR Scanner	Camera and NFC hardware access	Viewfinder overlay, scan result card, pairing confirmation
Notification Settings	iOS/Android notification channel management	Per-category toggles, quiet hours, critical alert permission
VPN Configuration	WireGuard profile installation	VPN status, connect/disconnect, profile management
Offline Dashboard	Must work without server connection	Cached device list, stale indicators, command queue
Watch App	Entirely native watchOS/Wear OS	Scene grid, device toggles, complications
App Settings	Native preferences, storage, permissions	Account, security, notifications, location, about

5.3 Navigation Structure

The app uses a bottom navigation bar with 5 primary destinations:

Tab	Content	Source
Home	3D/2D dashboard with device overview	WebView (ASCIA SvelteKit)
Cameras	Live camera grid, NVR timeline, clips	Hybrid (native player + WebView controls)
Scenes	Quick scene activation tiles	Native Flutter (works offline)
Alerts	Notification history and event log	Hybrid (WebView list + native notification bridge)
Settings	Account, security, permissions, VPN, about	Native Flutter

Section 6 — Development Roadmap

6.1 Phase Alignment with ASCIA OS

The mobile app development is synchronized with the ASCIA OS roadmap. The app cannot deliver features that the server does not yet support.

Phase	Timeline	Mobile App Deliverables	Depends On (ASCIA OS)
Phase 1 — MVP	Months 3-6	WebView dashboard, authentication, basic notifications, geofencing, local network discovery, offline cache	ASCIA OS MVP: frontend, device-manager, NVR basic, backup
Phase 2 — Advanced	Months 7-10	LiDAR scanning, NFC pairing, BLE presence, rich notifications with images and actions, camera intercom	ASCIA OS Phase 2: AI, mmWave, auto-blueprints, self-healing
Phase 3 — Commercial	Months 11-14	Apple Watch + Wear OS, UWB precision, full offline mode, App Store + Google Play deployment	ASCIA OS v1.0: full platform

6.2 MVP Scope (What Is Included)

- Flutter project setup with clean architecture, Riverpod, GoRouter.
- WebView integration with flutter_inappwebview, JS-Dart bridge, token injection.
- Authentication: biometric (Face ID / fingerprint), PIN, username/password.
- Server discovery via mDNS on local network.
- Basic push notifications (text + sound, no image or action buttons yet).
- Geofencing: home zone enter/exit with automation trigger.
- Offline cache: last-known device states displayed when server is unreachable.
- Camera live view: single camera HLS stream.
- App settings: account, logout, server connection info.

6.3 Post-MVP Scope (What Is Excluded from MVP)

- LiDAR 3D room scanning (Phase 2).
- NFC device pairing (Phase 2).
- BLE beacon presence broadcasting (Phase 2).
- UWB precision location (Phase 3).
- Rich notifications with images and action buttons (Phase 2).
- Two-way audio intercom (Phase 2).
- Apple Watch and Wear OS apps (Phase 3).
- Multi-camera grid view (Phase 2).

- NVR timeline scrubbing and clip export (Phase 2).
- Advanced offline mode with command queuing (Phase 3).

Section 7 — Testing Strategy

7.1 Test Coverage Requirements

Test Type	Target Coverage	Tools	Scope
Unit Tests	≥ 70%	flutter_test, mockito	Business logic, state management, data models, bridge handlers
Widget Tests	≥ 50%	flutter_test	Native UI screens (login, settings, scanner overlays)
Integration Tests	Critical paths	integration_test	Full flows: login → dashboard, NFC scan → pairing, geofence → notification
Platform Tests	All native plugins	XCTest (iOS), JUnit (Android)	LiDAR, NFC, BLE, UWB native code
E2E Tests	Top 10 user journeys	Patrol or integration_test	Complete app flows on real devices

7.2 Device Testing Matrix

The app must be tested on the following minimum device set:

Device	OS Version	Special Capabilities	Test Priority
iPhone 14 Pro	iOS 17	LiDAR, UWB, Face ID, NFC	Primary
iPhone 12	iOS 16	No LiDAR, Face ID, NFC	Primary
iPhone SE 3rd gen	iOS 15	No LiDAR, no UWB, Touch ID	Secondary
Samsung Galaxy S23	Android 14	UWB, NFC, fingerprint	Primary
Google Pixel 7	Android 13	NFC, fingerprint, no UWB	Primary
Samsung Galaxy A54	Android 12	NFC, fingerprint, no UWB, mid-range GPU	Secondary (performance test)
Apple Watch Series 8	watchOS 9	BLE, haptics	Phase 3
Samsung Galaxy Watch 5	Wear OS 4	BLE	Phase 3

7.3 Hardware Integration Testing

Since the app interacts with physical ASCIA hardware, the following tests require access to the ASCIA Lab IoT in Montreal (via the staging environment tunnel):

- Camera stream test: verify HLS playback from at least 2 physical ONVIF cameras.
- NFC pairing test: pair a physical ASCIA sensor using the NFC tag.

- Geofence test: verify automation trigger when phone enters/exits a real geofence zone.
- Notification delivery test: trigger a physical door sensor and verify push notification delivery time.
- Intercom test: verify two-way audio with a physical ASCIA doorbell camera.

Section 8 — App Store Deployment & Distribution

8.1 App Store Requirements

Requirement	iOS (App Store)	Android (Google Play)
Developer Account	Apple Developer Program (\$99 USD/year)	Google Play Developer (\$25 USD one-time)
App Review	Apple review (typically 24-48 hours)	Google review (typically 2-7 days for new apps)
Critical Alert Entitlement	Requires separate Apple approval with justification	Not required (high-priority channels available by default)
Background Location	Requires detailed justification in App Store Connect	Requires <code>ACCESS_BACKGROUND_LOCATION</code> with prominent in-app disclosure
NFC	Requires NFC entitlement in provisioning profile	Standard permission, no special approval
Privacy Nutrition Labels	Required: detailed data collection disclosure	Required: Data Safety section in Play Console
Age Rating	4+ (home automation, no objectionable content)	Everyone

8.2 CI/CD Pipeline

- Source code in the ASCIA GitHub Organization (ascia-mobile repository).
- GitHub Actions workflow: lint (Dart analyzer) → unit tests → widget tests → build iOS + Android.
- Automated builds: each merge to develop triggers a build. Tags (v*.*.*) trigger release builds.
- iOS: Fastlane for TestFlight distribution (internal testing) and App Store submission.
- Android: Fastlane for Google Play internal testing track and production submission.
- Code signing: iOS certificates and Android keystore stored in GitHub Secrets (encrypted).
- Beta distribution: TestFlight (iOS) and Google Play Internal Testing (Android) for ASCIA team and pilot customers.

8.3 Update Strategy

- The mobile app follows a separate update cycle from ASCIA OS.
- The app must maintain backward compatibility with ASCIA OS versions up to 2 major versions behind.
- API versioning: the app negotiates the API version with the server on connection and adapts its behavior accordingly.

- Force update: if a critical security issue is found, the app can display a mandatory update screen blocking access until the update is installed.
- Release cadence: monthly for minor updates, quarterly for major features, immediate for security patches.

Section 9 — Glossary

Term	Definition
ASCIA OS	The ASCIA home automation operating system based on Home Assistant OS, installed on the ASCIA server.
HA Core	Home Assistant Core — the Python backend engine managing entities, states, and automations.
Flutter	Google's cross-platform UI framework for building natively compiled applications from a single codebase.
Dart	The programming language used by Flutter.
Platform Channel	Flutter's mechanism for calling native Swift/Kotlin code from Dart.
flutter_inappwebview	Flutter plugin for embedding a WebView with full JavaScript bridge capabilities.
Riverpod	State management library for Flutter providing compile-time safe dependency injection.
GoRouter	Declarative routing package for Flutter with deep link support.
ARKit	Apple's augmented reality framework with LiDAR scanner support (iOS).
ARCore	Google's augmented reality framework with Depth API (Android).
GLB / GLTF	Standard 3D model format used for ASCIA 3D plans.
WebRTC	Real-time communication protocol used for two-way audio intercom.
HLS	HTTP Live Streaming — adaptive video streaming protocol used for camera feeds.
BLE	Bluetooth Low Energy — used for proximity detection and room-level presence.
UWB	Ultra-Wideband — radio technology for centimeter-accurate indoor positioning.
Geofencing	Location-based trigger system using GPS to detect entry/exit from defined geographic areas.
APNs	Apple Push Notification service — Apple's push notification delivery system.
FCM	Firebase Cloud Messaging — Google's push notification delivery system.
mDNS	Multicast DNS — protocol for discovering devices on a local network without a central DNS server.
WireGuard	Modern VPN protocol used for secure remote access to the ASCIA OS server.
NDEF	NFC Data Exchange Format — standard format for NFC tag payloads.

ASCIA Smart Home Systems

Specifications — ASCIA Mobile Application — Version 1.0
Confidential Document — © 2026 ASCIA Inc. — All rights reserved